Proof of Human Intent: Cryptographically Verifiable Human Approval for AI-Driven Development

Ikko Eltociear Ashimine
Independent Researcher
ORCID: 0000-0002-3576-6677

***Abstract*—With the rise of autonomous AI agents capable of proposing and executing software changes, traditional human-in-the-loop assumptions no longer hold. As these agents increasingly automate code generation, review, and deployment, the provenance of "human intent"—the accountability behind critical decisions—is becoming obscured. Current systems lack mechanisms to cryptographically verify that a human—rather than an automated process—authorized specific actions such as merging pull requests or deploying to production. We present Proof of Human Intent (PoHI), a protocol that leverages zero-knowledge proofs of personhood (e.g., World ID) and on-chain transparency logs to create tamper-evident, machine-verifiable records of human approval. The architecture can optionally integrate with decentralized identifiers (DIDs) and verifiable credentials (VCs) for enterprise authorization policies. Our architecture addresses three fundamental questions: (1) *who* approved an action (unique human verification), (2) *what* was approved (cryptographic binding to specific commits), and (3) *when* it was approved (immutable timestamping). We implement a proof-of-concept integration with GitHub Actions and demonstrate that PoHI prevents unauthorized automated merges with negligible latency overhead (<2 seconds total machine time). The reference implementation is available at https://github.com/pohi-protocol/pohi.**

***Index Terms*—AI agents, human-in-the-loop, proof of personhood, software supply chain security, zero-knowledge proofs, decentralized identity, AI governance, human approval**

I. Introduction

The rapid advancement of AI-driven development tools has fundamentally transformed software development workflows. Tools such as GitHub Copilot [10], Claude Code, and autonomous coding agents can now generate, review, and even propose merging code changes with minimal human intervention. While this automation dramatically increases developer productivity, it raises critical questions about accountability: *Who approved this code? Was it a human or an AI? Can we prove it?*

Consider a scenario where an AI agent autonomously creates a pull request, reviews it using another AI system, and merges it into production—all without meaningful human oversight. In 2024, GitHub Copilot assists in over 46% of code written by developers using the tool. By 2025, autonomous coding agents like Devin, Claude Code Agent, and GPT Engineer can complete entire development tasks end-to-end. This trend points toward a future where AI systems not only write code but also manage significant portions of the software lifecycle.

The accountability gap becomes critical when we consider:

- **Security incidents**: If malicious code is introduced, who is responsible?
- **Regulatory compliance**: Industries like finance and healthcare require human approval for critical changes.
- **Audit requirements**: SOC 2, ISO 27001, and similar frameworks mandate traceable approval processes.
- **Legal liability**: Contracts may require human sign-off for software releases.

Current solutions rely on social conventions (e.g., requiring reviews) or process controls (e.g., protected branches), but these are easily circumvented and lack cryptographic verifiability [9]. There is no mechanism to prove, after the fact, that a genuine human—not a bot or automated system—approved a specific action.

A. Contributions

This paper makes the following contributions:

- **Novel Problem Identification**: We identify the "human approval verification gap"—the inability to cryptographically prove that a human (rather than an automated system) authorized a specific software action. This gap is not addressed by existing supply chain security frameworks such as SLSA, Sigstore, or SCITT, which focus on artifact integrity rather than human verification.
- **First Integrated Protocol**: We propose PoHI, the first protocol that combines zero-knowledge proofs of personhood with software supply chain attestations, enabling cryptographically verifiable human approval without revealing user identity.
- **Formal Security Analysis**: We define formal security properties (unforgeability, replay resistance, Sybil resistance, and binding) and provide proofs under standard cryptographic assumptions.
- **Practical Implementation**: We provide an open-source reference implementation including a GitHub Action, CLI tool, and smart contracts, demonstrating negligible latency overhead (<2 seconds) for real-world CI/CD integration.

II. Background and Related Work

A. AI Agents in Software Development

The evolution of AI in software development can be characterized in three phases:

Phase 1 (2021-2023): Completion-based assistants. GitHub Copilot and similar tools provide inline code suggestions. The human developer remains in control of all actions.

Phase 2 (2024-2025): Agentic assistants. Tools like Claude Code, Cursor, and Windsurf can execute multi-step tasks, run commands, and modify files autonomously within developer-defined boundaries.

Phase 3 (2025+): Autonomous agents. Systems like Devin and OpenAI's Operator can complete entire tasks independently, including creating pull requests and interacting with external services.

This progression shifts developers from "implementers" to "approvers," creating an urgent need for verifiable human oversight.

B. Proof of Personhood

Proof of Personhood (PoP) systems aim to verify that an entity is a unique human without revealing their identity. While PoHI is designed to be provider-agnostic, we use World ID as a concrete instantiation in this work due to its strong cryptographic guarantees. Notable approaches include:

World ID [1], [2] uses iris biometrics via specialized hardware (Orb) to create a unique identifier. Zero-knowledge proofs allow users to prove their humanity without revealing biometric data. World ID provides both "Device" level (phone-based) and "Orb" level (biometric) verification.

BrightID uses a social graph approach where existing verified humans vouch for new members. While more accessible, it is more susceptible to Sybil attacks through collusion.

Gitcoin Passport aggregates multiple identity signals (social accounts, on-chain activity) into a "humanity score." This provides gradual verification but lacks the binary uniqueness guarantee of biometric systems.

We use World ID as a concrete PoP provider in our reference implementation due to its strong Sybil resistance and privacy-preserving properties through zero-knowledge proofs.

1) Provider-Agnostic Requirements: Although World ID is used as a concrete instantiation, PoHI is designed to be provider-agnostic. The protocol requires any proof-of-personhood system to satisfy three properties:

1) **Uniqueness**: Each human can obtain at most one valid credential.
2) **Unlinkability**: Proofs for different actions cannot be correlated to the same individual.
3) **Binding**: Proofs can be cryptographically bound to arbitrary signals (e.g., commit hashes).

Any PoP scheme meeting these properties—including future systems based on social graphs, behavioral biometrics, or hardware attestation—can be substituted without changes to the core PoHI protocol.

C. Software Supply Chain Security

Recent initiatives address software supply chain integrity:

SCITT (Supply Chain Integrity, Transparency, and Trust) [5] is an IETF draft architecture for maintaining append-only transparency logs of supply chain claims. It provides a framework for recording and verifying attestations about software artifacts.

Sigstore [6] enables keyless code signing using OIDC identity providers. It provides transparency logs (Rekor) for signatures but focuses on cryptographic identity rather than human verification.

SLSA (Supply-chain Levels for Software Artifacts) defines a framework for ensuring artifact integrity through provenance. However, it does not address the "human approval" aspect of the supply chain.

PoHI complements these systems by adding a human verification layer that can be integrated with existing supply chain security infrastructure.

D. Decentralized Identity

The W3C standards for Decentralized Identifiers (DIDs) [3] and Verifiable Credentials (VCs) [4] provide a foundation for self-sovereign identity. DIDs enable identifier creation without a central authority, while VCs allow claims about subjects to be cryptographically verified.

PoHI leverages these standards for its Authority Layer, enabling organizations to issue credentials specifying who can approve what types of actions.

E. AI Agent Authorization

South et al. [7] propose "Authenticated Delegation" for AI agents, introducing the concept of capability delegation from humans to AI systems. Their work focuses on what AI agents are authorized to do.

PoHI addresses a complementary problem: verifying that a human actually approved an action, regardless of whether AI agents were involved in the execution. While their approach defines "what can be done," PoHI proves "a human approved this."

F. Comparison with Existing Approaches

Table I summarizes the key differences between PoHI and existing supply chain security frameworks.

Existing supply chain security frameworks (SLSA, Sigstore, SCITT) provide artifact integrity and provenance but do not verify that a *human* performed the action. Proof-of-personhood systems (BrightID, Gitcoin Passport) verify human uniqueness but lack binding to specific software artifacts (commits, releases). PoHI uniquely combines both properties: cryptographic proof of human identity bound to specific software artifacts.

TABLE I
COMPARISON OF SUPPLY CHAIN SECURITY APPROACHES

System	Human Verify	Artifact Binding	Sybil Resist
SLSA [11]	×	✓	×
Sigstore [6]	×	✓	×
SCITT [5]	×	✓	×
BrightID	✓	×	Weak
Gitcoin Passport	✓	×	Heuristic
PoHI (Ours)	✓	✓	**Strong**

III. Threat Model

A. System Model

We consider a software development environment with the following components:

- **Developers**: Human actors who write and review code.
- **AI Agents**: Automated systems capable of generating code and creating pull requests.
- **Repository**: A Git-based version control system.
- **CI/CD Pipeline**: Automated build and deployment infrastructure.

B. Adversary Capabilities

We assume an adversary who can:

- Control AI agents with repository access.
- Create commits and pull requests programmatically.
- Attempt to forge or replay approval attestations.
- Create multiple fake identities (Sybil attack).
- **Insider threat**: A legitimate user with valid World ID credentials who intentionally approves malicious changes. PoHI provides cryptographic evidence of who approved, enabling post-incident attribution, but does not prevent authorized insiders from acting maliciously.

C. Security Goals

PoHI aims to ensure:

1) **Human Verification**: Actions are verifiably approved by unique humans.
2) **Binding**: Approval is bound to specific code states.
3) **Non-repudiation** (cryptographic): Approvals cannot be denied without compromising cryptographic assumptions.
4) **Tamper Evidence**: Modifications to records are detectable.

D. Trust Assumptions

The security of PoHI relies on the following trust assumptions:

- **World ID System**: We assume the World ID ZK-SNARK system is sound and the biometric enrollment process (Orb) provides strong uniqueness guarantees. Device-level verification provides weaker assurance.
- **Verification API**: The reference implementation uses the World ID verification API as a trusted third party. For higher assurance, on-chain verification against World ID's verification contracts can eliminate this trust assumption at the cost of increased latency and gas costs.
- **Blockchain Integrity**: We assume the underlying blockchain (World Chain) provides standard properties: immutability, censorship resistance, and reliable timestamping.

E. Scope and Limitations

It is important to distinguish between *cryptographic authorization* and *semantic understanding*. PoHI guarantees that a specific signal (commit hash) was approved by a unique human identity. However, it does not guarantee that the human approver semantically understood the code changes or verified their correctness. Social engineering attacks where a human is coerced or tricked into scanning the QR code for a malicious commit are considered out of scope for the cryptographic protocol, though UI mitigations (e.g., displaying commit details clearly before approval) are recommended. Additionally, PoHI does not prevent a malicious insider with legitimate World ID credentials from approving harmful changes—organizational access controls and code review processes remain complementary safeguards. PoHI provides cryptographic evidence of human approval but does not replace legal or organizational authorization frameworks; rather, it offers verifiable records that can support existing governance and compliance requirements. PoHI explicitly separates proof of human presence from authorization semantics; authority is enforced through external policy and governance layers.

IV. Proposed Architecture

A. Overview

PoHI consists of four layers that work together to create verifiable human approval records:

1) **Identity Layer**: Verifies the approver is a unique human using World ID.
2) **Authority Layer**: Manages permissions using DIDs and VCs (optional).
3) **Attestation Layer**: Records events on-chain for immutable transparency.
4) **Integration Layer**: Connects with Git workflows.

Figure 1 illustrates the layered architecture and data flow between components.

The core data structure is the `HumanApprovalAttestation`, which binds:

- A proof of human identity (World ID nullifier hash)
- A specific software artifact (commit SHA)
- An action type (merge, deploy, release)
- A timestamp (block time or signed timestamp)

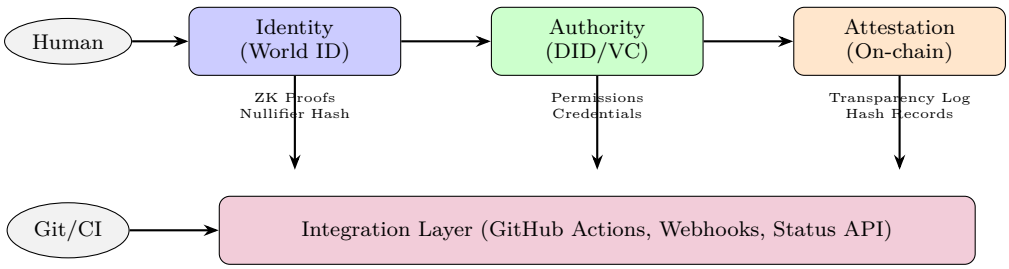

Fig. 1. PoHI Four-Layer Architecture. Human approval flows through identity verification (Layer 1), optional authority checks (Layer 2), and attestation recording (Layer 3), all connected to developer workflows via the integration layer (Layer 4).

B. Identity Layer

The Identity Layer leverages World ID's zero-knowledge proof system. When a user approves an action:

1) The system generates a `signal` by hashing the repository and commit SHA:
   `signal = SHA256(repository || ":" || commit_sha)`
2) The user scans a QR code with the World App, which generates a ZK proof binding their World ID to this signal.
3) The proof includes a `nullifier_hash` unique to this user-action combination, preventing the same human from approving the same commit twice within the same repository and action scope, while preserving privacy.

The signal is deliberately limited to repository and commit SHA to provide a minimal, unambiguous cryptographic binding. Additional metadata such as action type or timestamp is recorded in the attestation structure rather than in the signal itself, allowing the same ZK proof to be verified independently of attestation formatting choices. The nullifier is domain-separated by the World ID application identifier, preventing cross-application correlation.

Verification levels provide flexibility:

- **Device**: Phone-based verification (lower assurance)
- **Orb**: Biometric verification (higher assurance)

C. Authority Layer

The Authority Layer (optional) enables organizations to define who can approve what:

- **DIDs** identify organizations and individuals
- **VCs** specify approval authorities (e.g., "Alice can approve production deployments for repo X")

This layer is not required for basic operation but enables enterprise use cases such as role-based approval policies.

D. Attestation Layer

The Attestation Layer creates and stores attestation records. Two hash algorithms are used:

- **SHA-256**: Protocol-standard hash for off-chain storage and interoperability
- **Keccak-256**: EVM-native hash for on-chain recording

On-chain storage on World Chain provides:

- Immutability and tamper evidence
- Public verifiability
- Censorship resistance
- Timestamp through block inclusion

The smart contract enforces:

- Uniqueness of attestation hashes
- Prevention of duplicate approvals (same nullifier + commit)
- Revocation capability for the original approver or admins

E. Integration Layer

The Integration Layer connects PoHI to developer workflows:

GitHub Action: A GitHub Action that can be added to any repository to enforce human approval before merging. When triggered:

1) Creates an approval request with the commit SHA
2) Posts a QR code to the PR for World ID verification
3) Waits for human approval via the World App
4) Records the attestation and updates PR status

Figure 2 illustrates the complete verification flow from pull request creation to merge enablement.

CLI Tool: A command-line interface for requesting and verifying approvals outside of CI/CD contexts.

SDK: TypeScript libraries for integrating PoHI into custom applications.

V. Implementation

We implement PoHI as a modular TypeScript library with the following packages:

- `pohi-core`: Chain-neutral types, validation, and SHA-256 hashing (zero dependencies)
- `pohi-evm`: EVM utilities for Keccak-256 and on-chain encoding
- `pohi-sdk`: Client for World Chain interaction
- `pohi-cli`: Command-line tool
- `pohi-action`: GitHub Action for CI/CD integration
- `pohi-contracts`: Solidity smart contracts (Foundry)

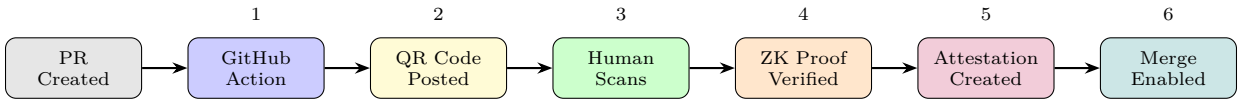

Fig. 2. Verification Flow. (1) GitHub Action triggers on PR, (2) posts QR code comment, (3) human scans with World App, (4) ZK proof verified against World ID API, (5) attestation created and stored (on-chain recording is optional), (6) PR status updated to enable merge.

The core library has zero runtime dependencies, enabling use in constrained environments. The modular design allows users to adopt only the components they need.

A. Attestation Data Model

The attestation JSON structure contains the following fields:

- **version**: Schema version (e.g., "1.0")
- **type**: "HumanApprovalAttestation"
- **subject**: Object containing:
  - *repository*: Target repository (e.g., "org/repo")
  - *commit_sha*: Git commit hash
  - *action*: Action type (PR_MERGE, DEPLOY, etc.)
- **human_proof**: Object containing:
  - *method*: Verification method ("world_id")
  - *verification_level*: "device" or "orb"
  - *nullifier_hash*: Unique identifier preventing replay
- **timestamp**: ISO 8601 timestamp
- **proof**: Cryptographic signature (Ed25519 JWS)

VI. Evaluation

A. Formal Security Analysis

We formally define the security properties of PoHI and provide proofs under standard cryptographic assumptions.

1) Security Definitions: Let $\mathcal{A}$ denote an attestation, H a cryptographic hash function (SHA-256), π a World ID zero-knowledge proof, and ν the nullifier hash derived from the proof.

Definition 1 (Unforgeability). A PoHI attestation scheme is *unforgeable* if no probabilistic polynomial-time (PPT) adversary $\mathcal{E}$ can produce a valid attestation $\mathcal{A}^*$ for a commit c without possessing a valid World ID proof π for signal $s = H(\text{repo}\|c)$, except with negligible probability.

Definition 2 (Replay Resistance). A PoHI attestation scheme is *replay-resistant* if a valid attestation $\mathcal{A}$ for commit c_1 cannot be used to verify approval for a different commit c_2 where $c_1 \neq c_2$.

Definition 3 (Sybil Resistance). A PoHI attestation scheme is *Sybil-resistant* if each unique human can produce at most one valid attestation per (repository, commit, action) tuple.

Definition 4 (Binding). A PoHI attestation $\mathcal{A}$ is *bound* to subject S if any modification to S results in verification failure.

2) Security Proofs: **Theorem 1 (Unforgeability).** PoHI attestations are unforgeable under the soundness of the World ID ZK-SNARK system [8] and collision resistance of SHA-256.

Proof. Assume an adversary $\mathcal{E}$ can forge an attestation $\mathcal{A}^*$ for commit c without a valid World ID proof. The attestation verification requires: (1) a valid ZK proof π that verifies against the World ID verification contract, and (2) the proof's signal matches $H(\text{repo}\|c)$. By the soundness property of the ZK-SNARK, $\mathcal{E}$ cannot produce a valid π without knowledge of the witness (World ID credentials). Thus, forging $\mathcal{A}^*$ requires breaking either ZK-SNARK soundness or SHA-256 collision resistance, both assumed to be computationally infeasible. □

Theorem 2 (Replay Resistance). PoHI attestations are replay-resistant under collision resistance of SHA-256.

Proof. The signal in a World ID proof is computed as $s = H(\text{repo}\|c)$. For an attestation valid for c_1 to be replayed for c_2, we require $H(\text{repo}\|c_1) = H(\text{repo}\|c_2)$ with $c_1 \neq c_2$. This constitutes a collision in SHA-256, which is computationally infeasible under standard assumptions. □

Theorem 3 (Sybil Resistance). PoHI provides Sybil resistance bounded by the uniqueness guarantees of the underlying World ID system.

Proof. The nullifier hash ν is derived deterministically from the user's World ID identity and the application-specific external nullifier. For a given (repository, action) scope, each unique World ID produces a unique ν. The on-chain registry rejects duplicate nullifiers. Thus, creating multiple attestations requires either (1) multiple World ID identities (prevented by biometric uniqueness at Orb level), or (2) nullifier collision (computationally infeasible). □

Theorem 4 (Binding). PoHI attestations are cryptographically bound to their subjects.

Proof. The attestation hash is computed as $h = H(\mathcal{A})$ where $\mathcal{A}$ includes all subject fields (repository, commit_sha, action). Any modification to these fields changes h, causing verification failure against the recorded hash. □

3) Attack Vector Analysis: Table II summarizes the security guarantees against specific attack vectors.

B. Performance Evaluation

We measured the end-to-end latency of the approval verification flow using the reference implementation. Table III shows the breakdown of processing time for a typical validation request.

The primary latency factor is the external World ID verification API call (∼1.2s), which is acceptable for

TABLE II
SECURITY ANALYSIS AGAINST ATTACK VECTORS

Attack	Mitigation	Guarantee
Forgery	Theorem 1	Computational
Replay	Theorem 2	Computational
Sybil	Theorem 3	Biometric + Crypto
Tampering	Theorem 4	Computational
Front-running	On-chain ordering	Blockchain

TABLE III
OPERATION LATENCY (REFERENCE IMPLEMENTATION)

Operation	Time (ms)	Std Dev (ms)
Attestation Construction	<5	±1
Hash Computation (SHA-256)	<1	±0.1
World ID Proof Verification (API)	1,200	±300
GitHub Status Update	250	±50
On-chain Recording (optional)	3,000	±500
Total Machine Time	**∼1,500**	-

asynchronous pull request workflows. The computational overhead for cryptographic binding (SHA-256 hashing and signature verification) is negligible (< 5ms) on standard CI/CD runners. Human interaction time (scanning QR code with World App) is excluded from machine time as it varies by user but typically requires 5–15 seconds. The protocol overhead introduced by PoHI itself remains negligible, independent of the underlying proof-of-personhood provider.

C. Gas Costs

On-chain operations incur gas costs on World Chain:

TABLE IV
ESTIMATED GAS COSTS

Operation	Gas Units
recordAttestation	∼150,000
revokeAttestation	∼30,000
isValidAttestation (view)	0

At typical World Chain gas prices, recording an attestation costs less than $0.01 USD.

These results indicate that PoHI can be integrated into existing CI/CD pipelines with negligible overhead, making human approval verification practical for real-world software development workflows.

Reproducibility. The reference implementation is publicly available at https://github.com/pohi-protocol/pohi under the Apache 2.0 license. Performance measurements were conducted on GitHub Actions runners (Ubuntu 22.04, 2-core CPU) using the deployed World Chain Sepolia testnet. The demo application is accessible at https://pohi-demo.vercel.app for interactive verification flow demonstration.

VII. DISCUSSION

A. Design Philosophy: Presence vs. Understanding

A natural critique of PoHI is that it proves human *presence* but not human *understanding*. This separation is intentional. PoHI verifies that a unique human approved a specific action at a specific time—cryptography can provide this guarantee. However, whether that human semantically understood the code changes, verified their correctness, or made an informed decision remains a socio-technical responsibility that no cryptographic protocol can enforce.

We argue this separation is appropriate: (1) it matches real-world accountability models where signatures attest to approval, not comprehension; (2) it enables automation of the verifiable component while leaving judgment to humans and organizations; and (3) attempting to cryptographically verify "understanding" would require solving fundamentally unsolvable problems in human cognition assessment.

PoHI provides the cryptographic foundation; organizational policies, code review practices, and user interface design must address the understanding component.

B. Limitations

- **World ID Orb Availability**: The highest assurance level (Orb verification) requires physical access to specialized hardware, which has limited global availability. Device-level verification provides a fallback but with weaker Sybil resistance.
- **Verification Level Trade-offs**: Device-level verification relies on phone attestation rather than biometrics, making it susceptible to sophisticated device spoofing. Organizations requiring strong guarantees should mandate Orb-level verification.
- **API Trust Dependency**: The reference implementation relies on World ID's verification API, introducing a trusted third party. While on-chain verification is possible, it increases latency and costs.
- **Privacy in Transparency Logs**: On-chain attestations are publicly visible. While nullifier hashes preserve identity privacy, the fact that *someone* approved a specific commit is public. Organizations requiring private approvals must use off-chain storage with selective disclosure.
- **Adoption Barriers**: Integrating PoHI requires developers to install the World App and complete verification, adding friction to existing workflows. Organizational rollout requires change management.

C. Future Work

- Policy-as-code integration.
- Cross-organization trust federation.
- Privacy-preserving audit mechanisms.
- Support for emerging standards such as Verifiable Credentials v2.0.

- Broader interoperability across proof-of-personhood providers.
- Formal compatibility specification for third-party implementations.

We encourage the development of PoHI-compatible implementations that preserve the core principle of cryptographically verifiable human intent while adapting to diverse architectural requirements. The reference implementation and compatibility guidelines are available at https://github.com/pohi-protocol/pohi.

VIII. Conclusion

As AI agents become increasingly capable of autonomous software development, the ability to verify human approval becomes critical for accountability and security. We presented Proof of Human Intent (PoHI), a protocol combining zero-knowledge proofs, decentralized identity, and transparency logs to create verifiable records of human approval.

PoHI shifts accountability in AI-driven development from implicit trust to cryptographically verifiable intent, establishing a foundation for human oversight in an increasingly automated software ecosystem.

Acknowledgments

The author acknowledges the World Foundation for publicly documenting the World ID protocol. This work was inspired by discussions in the broader Web3 and decentralized identity communities.

References

[1] World Foundation, "World ID: Privacy-Preserving Proof of Personhood," 2024. [Online]. Available: https://docs.world.org/world-id

[2] M. Blania, A. Tromer, et al., "Worldcoin: A Global Identity and Financial Network," Worldcoin Foundation Whitepaper, 2023.

[3] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022.

[4] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model v2.0," W3C Recommendation, May 2025. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/

[5] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains," IETF Internet-Draft draft-ietf-scitt-architecture, 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/

[6] L. Newman, B. Beyer, et al., "Sigstore: Software Artifact Signing and Verification," in Proc. USENIX Security Symposium, 2022.

[7] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," arXiv:2501.09674, 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[8] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge," in Proc. CRYPTO, 2013.

[9] C. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," in Proc. IEEE S&P, 2023.

[10] GitHub, "GitHub Copilot: Your AI Pair Programmer," 2024. [Online]. Available: https://github.com/features/copilot

[11] Google, "SLSA: Supply-chain Levels for Software Artifacts," 2023. [Online]. Available: https://slsa.dev